



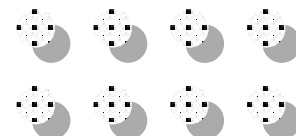
Classification Models with Scikit-learn

Eng | Mahmoud Tariq

09/16/2025

Course Syllabus

- Intro to Machine Learning & Environment Setup
- Foundations: Key Linear Algebra & Stats + Hands-on NumPy
- Data Handling & Visualization + Hands-on Pandas
- The ML Workflow + Linear Regression with Scikit-learn
- **Classification Models (Logistic Regression, KNN) with Scikit-learn**
 - **Task: Solve session 4's 'task.ipynb' notebook**
- Model Evaluation & Improvement (Metrics, Overfitting, Cross-Validation)
- Mini-Project: Putting it all together with Scikit-learn
- Introduction to Neural Networks
- Building a Neural Network with a Framework
- Course Project Description + Where to Go From Here



Classification

- What is Classification?
- Classification models
- Sci-kit Learn Hands-on



Classification?

- A classification problem is trying to identify the “class” of an instance in the dataset. (e.g. a dataset pictures of fruits, trying to identify each fruit is a classification problem)
- There are mainly two types of classification:
 - Binary classification
 - Multi-class classification

Classification: Binary?

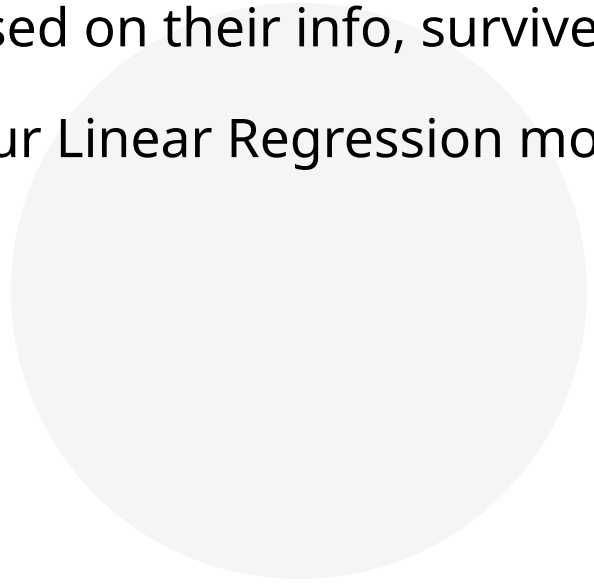
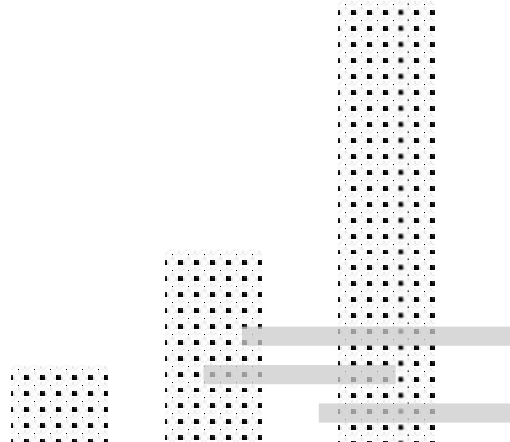
- A Binary Classification problem is when each instance of the dataset falls in one of two classes.
- Example: on the titanic dataset, if we try to predict on the 'survived' column, the result could either be 1 meaning the person survived or 0 meaning they didn't.
- The end result the model always yeilds is either 1 or 0 but we translate that ourselves to the actual classes.

Classification: Multi-class?

- A Multi-class Classification problem is when each instance of the dataset falls in one of N number of classes.
- Example: Imagine a dataset of images of fruits, each image corresponds to one fruit (i.e. class), so one image could be an apple another could be a banana or a pineapple.
- The model here returns a list of probabilities for each class, for example: an image of an apple return 80% for the apple class 5% for orange, 2% for pineapple, etc...

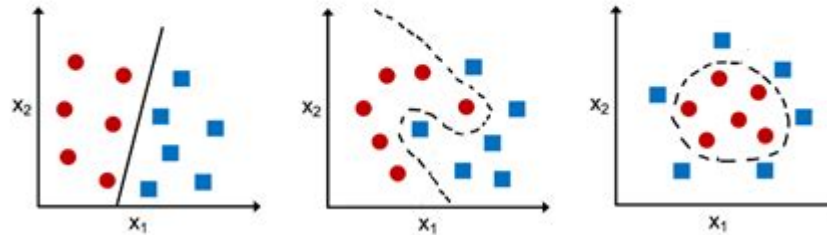


Classification: Multi-class?

- For now we will focus on solving a Binary classification problem on the titanic dataset, where we will try to predict whether each passenger, based on their info, survived or not.
 - But how will our Linear Regression model, handle this problem?
- 
- 

Classification: Models

- Let's try to define what we are trying to do, we want our model to predict either 0 or 1, not an arbitrary number. So how can we "limit" the output from our model to just 0 & 1?

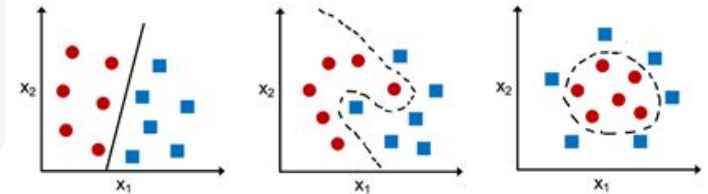


Classification: Models

- If we look back at our model definition, it doesn't define a limit to the output (i.e. it can produce values between $-\infty$ to $+\infty$), also it's output scales linearly which means it can only produce lines.

$$\begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}_{1 \times N} = \begin{bmatrix} x_1 & \cdots & x_M \end{bmatrix}_{1 \times M} \begin{bmatrix} w_{11} & \cdots & w_{1N} \\ \vdots & \ddots & \vdots \\ w_{M1} & \cdots & w_{MN} \end{bmatrix}_{M \times N} + \begin{bmatrix} b_1 \\ \vdots \\ b_N \end{bmatrix}_{1 \times N}$$

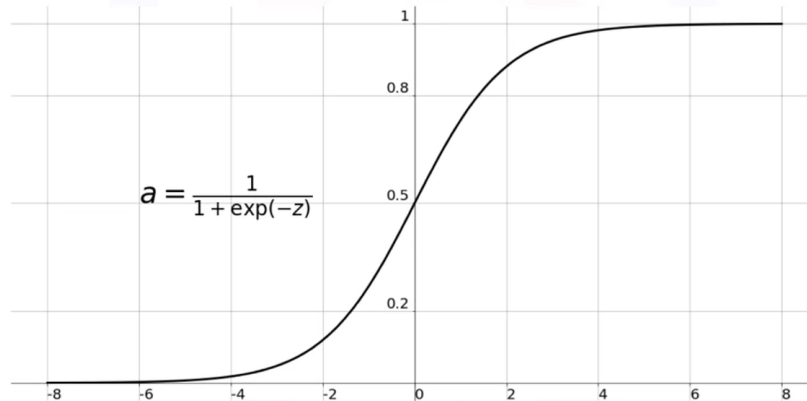
Note: the change in the Y vector output is changed to $1 \times N$ to denote N outputs are needed, also now the X and w matrices output: $1 \times M @ M \times N = 1 \times N$ matching the needed number of outputs also B bias vector is changed to match it.



Classification: Models

- To solve this we introduce a non-linear function to our model, for this specific case it's the sigmoid function (σ):
 - Notice the output is bounded between 1 & 0.

Sigmoid Function



Classification: Models

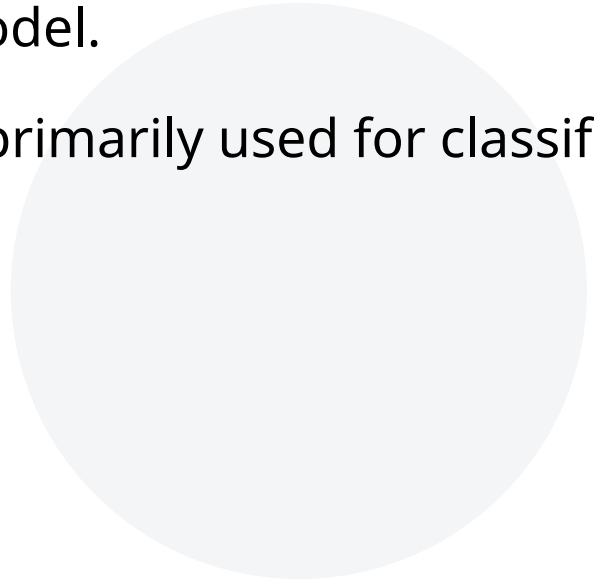
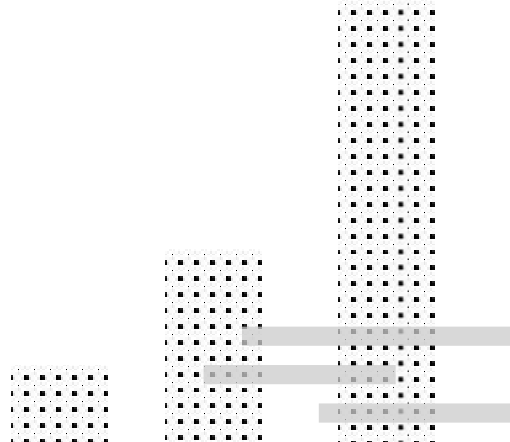
- So our new model definition would look like this:

$$\begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}_{1 \times N} = \sigma \left(\begin{bmatrix} x_1 & \cdots & x_M \end{bmatrix}_{1 \times M} \begin{bmatrix} w_{11} & \cdots & w_{1N} \\ \vdots & \ddots & \vdots \\ w_{M1} & \cdots & w_{MN} \end{bmatrix}_{M \times N} + \begin{bmatrix} b_1 \\ \vdots \\ b_N \end{bmatrix}_{1 \times N} \right)$$

Note: The entire output of the model is wrapped by the Sigmoid function (σ). If the output is a vector we simply apply the function to each element.



Classification: Models

- Since the sigmoid function is a logistical function directly applied to a regression model, we call this new model a Logistic Regression model.
 - This model is primarily used for classification.
- 
- 

Classification: Sci-kit Learn

- Similar to the LinearRegression class, Sci-kit learn provides a simple class for the Logistic Regression model appropriately named LogisticRegression:

```
from sklearn.linear_model import LogisticRegression  
  
model = LogisticRegression() # The model is used in the same way as the LinearRegression class.
```

Classification: Sci-kit Learn

- We will leave cleaning and preparing the data in 'code.ipynb', you can check that for extra explanation on what we did, but it is nothing new.
- Here is the dataset after cleaning:

```
final_titanic.head()
```

✓ 0.0s

	female	male	C	Q	S	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId											
1	False	True	False	False	True	0	3	22.0	1	0	7.2500
2	True	False	True	False	False	1	1	38.0	1	0	71.2833
3	True	False	False	False	True	1	3	26.0	0	0	7.9250
4	True	False	False	False	True	1	1	35.0	1	0	53.1000
5	False	True	False	False	True	0	3	35.0	0	0	8.0500

Classification: Sci-kit Learn

- Then we split it into X (input) and y (output):

```
y = final_titanic.pop("Survived") # this is the value we want to predict.  
X = final_titanic # all other columns are input to our model.  
  
X.shape, y.shape  
✓ 0.0s  
( (889, 10), (889,) )
```

Classification: Sci-kit Learn

- Then into train and test splits:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2) # We will use the same test size as last time.

print(f"Train dataset shapes: {X_train.shape}, {y_train.shape}")
print(f"Test dataset shapes: {X_test.shape}, {y_test.shape}")
```

✓ 0.0s

Train dataset shapes: ((711, 10), (711,))
Test dataset shapes: ((178, 10), (178,))

Classification: Sci-kit Learn

- Then into train and test splits:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2) # We will use the same test size as last time.

print(f"Train dataset shapes: {X_train.shape, y_train.shape}")
print(f"Test dataset shapes: {X_test.shape, y_test.shape}")
```

✓ 0.0s

Train dataset shapes: ((711, 10), (711,))
Test dataset shapes: ((178, 10), (178,))

Classification: Sci-kit Learn

- Then we'll use the same model we defined earlier:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression() # The model is used in the same way as the LinearRegression class.

5] ✓ 0.0s
```

Classification: Sci-kit Learn

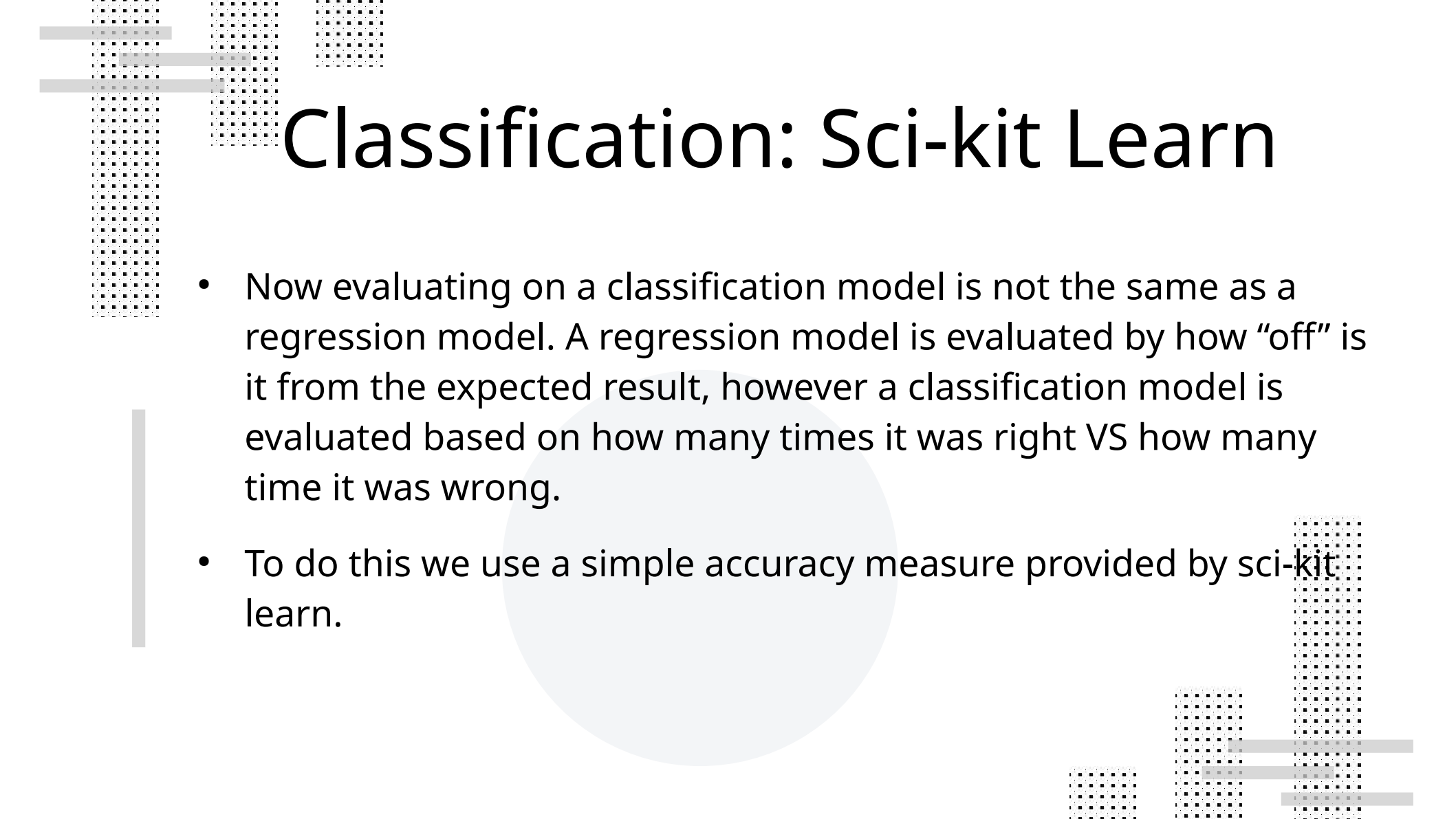
- And we will train it in the same way we did with LinearRegression:

```
# Then we use the same model we instantiated  
model.fit(X_train, y_train)  
✓ 0.0s
```

Classification: Sci-kit Learn

- Then we will predict on the test data:

```
# predict on the test data  
y_pred = model.predict(X_test)  
✓ 0.0s
```



Classification: Sci-kit Learn

- Now evaluating on a classification model is not the same as a regression model. A regression model is evaluated by how “off” is it from the expected result, however a classification model is evaluated based on how many times it was right VS how many time it was wrong.
- To do this we use a simple accuracy measure provided by sci-kit learn.

Classification: Sci-kit Learn

- To use it we do as follows:

```
from sklearn.metrics import accuracy_score

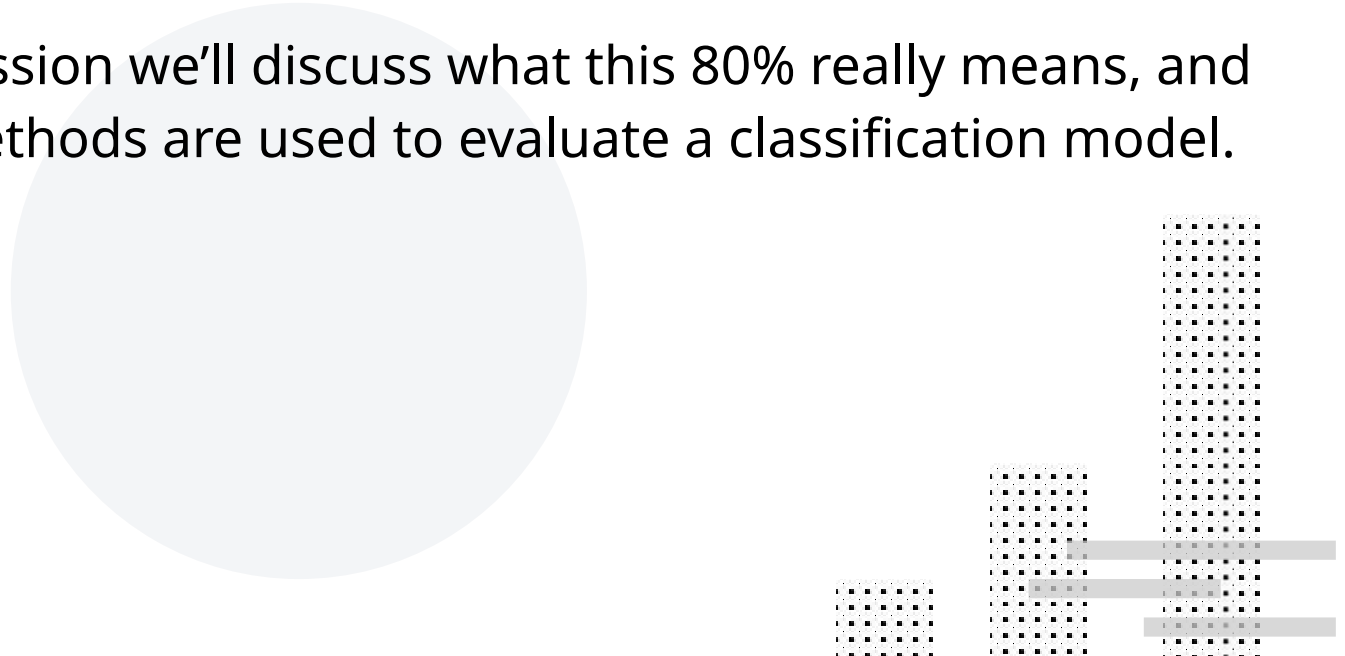
# we use it the same way we did with "MSE" (mean squared error) and "MAE" (mean absolute error)
accuracy = accuracy_score(y_test, y_pred)
print(f"We got: {accuracy*100:.2f}% right!")
```

✓ 0.0s

We got: 80.34% right!



Classification: Sci-kit Learn

- We didn't get 100% (All) of the data correct but we got ~80% right, which is impressive!
 - In the Next session we'll discuss what this 80% really means, and what other methods are used to evaluate a classification model.
- 

Clustering

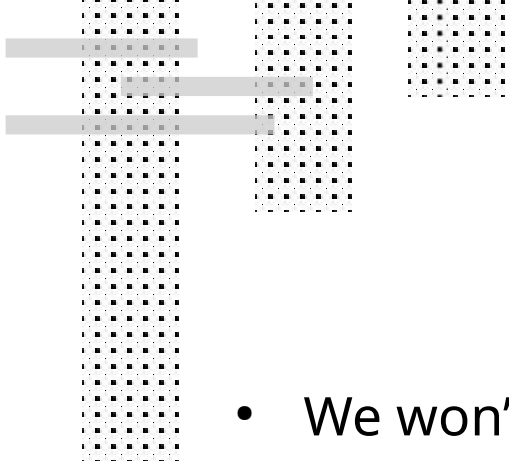
- What is Clustering?
- Clustering models
- Sci-kit Learn Hands-on



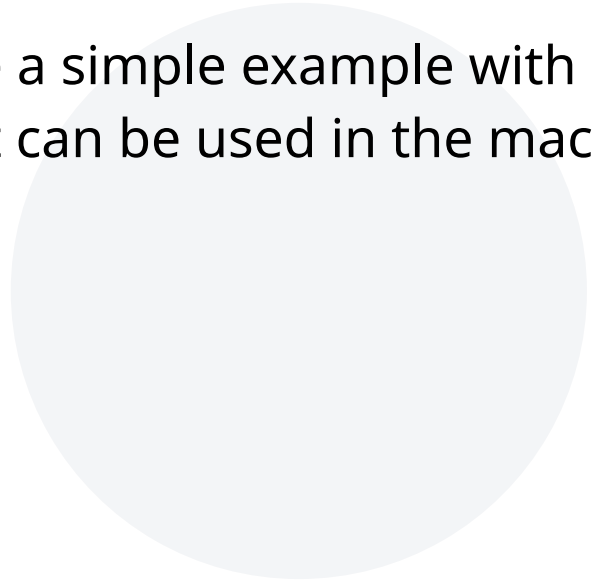
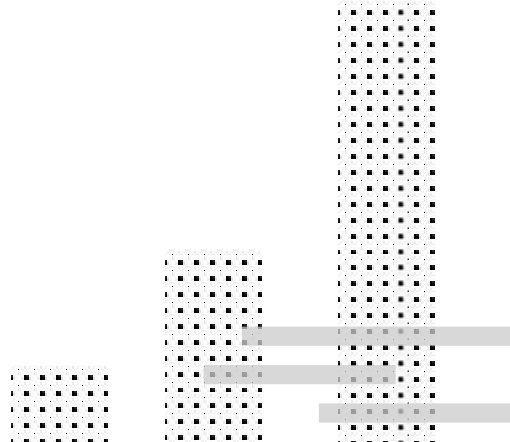
Clustering?

- Clustering is a different approach to the same problem of identifying the class of each instance, but instead of trying to learn a formula to predict the class, we try to simply group similar data points together.



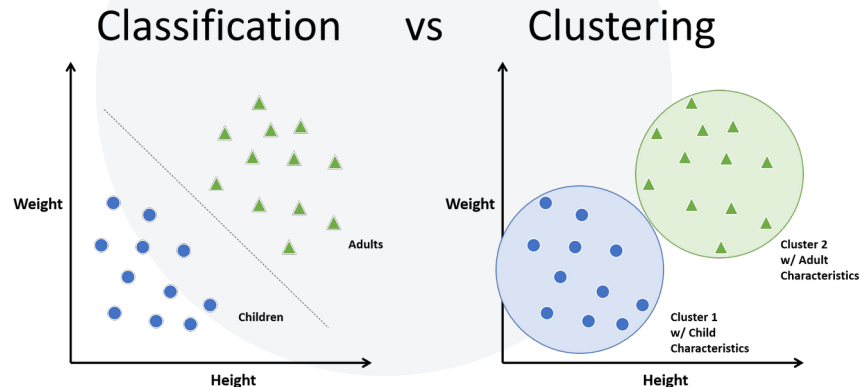


Clustering?

- We won't dive into the complexities of clustering models, as they are not built on the foundation we've laid so far.
 - So we will take a simple example with KNN clustering to show it's use and how it can be used in the machine learning workflow.
- 
- 

Clustering: KNN?

- K-Nearest Neighbors: is a non-formula based model, but based static coding logic. The K stands for how many “neighbors” the model compares to fit the current data point with. The “nearest neighbor” part stands for that a point in space’s neighbors are most likely in the same class.



Clustering: KNN?

- Sci-kit learn provides a KNN class to use the same way as the models we've used so far.

```
from sklearn.neighbors import KNeighborsClassifier  
  
knn = KNeighborsClassifier(n_neighbors=5) # 'n_neighbors' argument represents the 'K' we discussed
```

Clustering: KNN?

- Training & Prediction remain the same:

```
knn.fit(X_train, y_train) # we call the same '.fit()' method.  
  
knn_y_pred = knn.predict(X_test) # Same with the '.predict()' method.  
✓ 0.0s
```

Clustering: KNN?

- Evaluation:

```
knn_accuracy = accuracy_score(y_test, knn_y_pred)
print(f"We got: {knn_accuracy*100:.2f}% right!")
```

✓ 0.0s

We got: 71.35% right!

Clustering: KNN?

- We seem to have gotten a worse score than the LogisticRegression model, why is that?
- How do we know which models will perform better than other?
- Is this the only method for evaluation?
- In the next session we will try to answer these questions and introduce new models to compare to ours!

Session Task & What's Next

- Your Task: task.ipynb in session 5 folder
- Next Session: Model Evaluation & Improvement (Metrics, Overfitting, Cross-Validation)



Thank you.

